

3) System Topology

Last updated by | Mennell, Jacob | 19 May 2026 at 12:16 BST

System Topology

Purpose

This page describes the main deployed components in the solution and the runtime boundaries between presentation, API delivery, worker execution, storage, and messaging.

Topology Summary

The repository defines a containerized multi-service topology composed of:

- an NGINX-hosted frontend
- a FastAPI backend API
- a main background worker
- an ingestion background worker
- a PostgreSQL database with pgvector enabled
- a RabbitMQ broker
- a migrations container used to apply schema changes
- an optional beat service pattern that is present but not enabled by default in the local compose file

Component Responsibilities

Frontend

The frontend is built separately and served through NGINX. Its role is to provide the browser-based experience for navigating claims, extracted information, checks, and administrative pages.

API

The API is the synchronous integration and delivery surface. It exposes authenticated endpoints and acts as the primary entry point for client applications.

Main worker

The main worker executes orchestrated AI workflows, including extraction and checks. It is responsible for retrieval-backed runtime processing rather than direct user-facing request handling.

Ingestion worker

The ingestion worker processes documents into stored chunks and embeddings so that later workflows can search and reason over the resulting evidence set.

PostgreSQL with pgvector

The database holds application state, document records, chunk records, metadata, and vector embeddings used for retrieval.

RabbitMQ

RabbitMQ supports asynchronous job execution and decouples request-triggered workflows from background processing.

Migrations

The migrations container applies database schema updates before dependent services start.

Local Runtime Topology



flowchart LR

User[Browser User] --> FE[Frontend / NGINX]

FE --> API[FastAPI API]

API --> MQ[RabbitMQ]

API --> DB[(PostgreSQL + pgvector)]

MQ --> MW[Main Worker]

MQ --> IW[Ingestion Worker]

MW --> DB

IW --> DB

IW --> AI1[Azure Document Intelligence]

MW --> AI2[Azure Foundry / OpenAI]

Runtime Relationships

- The frontend depends on the API as its primary backend integration point.
- The API depends on the database for application data and can trigger background processing through the worker ecosystem.
- The workers depend on the broker for asynchronous task flow and on the database for persisted state and retrieval assets.
- The ingestion worker also depends on document parsing and embedding services.
- The main worker depends on retrieval-ready chunk data and AI model endpoints to execute workflows.

Design Implications

This topology separates:

- user-facing interaction from background AI execution
- document preprocessing from downstream reasoning workflows
- operational data storage from transient compute activity

That separation improves maintainability and allows scaling decisions to be made independently for ingestion, orchestration, and API delivery.